

XÁC THỰC VÀ PHÂN QUYỀN VỚI ASP.NET CORE IDENTITY

Bài trước, chúng ta đã hoàn thiện form quản trị. Tuy nhiên, hiện tại có một lỗ hổng bảo mật nghiêm trọng: **bất kỳ ai biết URL /Admin đều có thể vào và thay đổi dữ liệu sản phẩm!** Điều này là không thể chấp nhận được. Bài này, chúng ta sẽ triển khai **ASP.NET Core Identity**, một hệ thống bảo mật cấp độ doanh nghiệp, để "khóa" khu vực quản trị lại và chỉ cho phép những người có thẩm quyền truy cập.

MỤC TIÊU BÀI HỌC

Sau khi hoàn thành Bài học này, bạn sẽ có khả năng:

- **Phân biệt rành mạch** giữa Authentication (Xác thực) và Authorization (Phân quyền).
- **Tích hợp ASP.NET Core Identity** vào một dự án có sẵn bằng công cụ Scaffolding.
- **Sử dụng Migrations** để tạo và quản lý các bảng CSDL phức tạp cho hệ thống người dùng.
- **Bảo vệ các Controller hoặc Action** một cách hiệu quả bằng attribute [Authorize] và phân quyền dựa trên vai trò.

NỘI DUNG CHI TIẾT

10.1. Giới thiệu Authentication & Authorization - "Thẻ ID" và "Chìa khóa phòng"

Đây là hai khái niệm nền tảng, là "trụ cột" của bất kỳ hệ thống bảo mật nào và thường bị nhầm lẫn với nhau. Để xây dựng một ứng dụng an toàn, việc hiểu rõ sự khác biệt và mối quan hệ giữa chúng là vô cùng quan trọng.

10.1.1. Authentication (Xác thực) - "Bạn là ai?"

- **Là gì?** Authentication là quá trình xác minh **danh tính** của một người dùng, client, hoặc hệ thống, nhằm mục đích chứng minh họ đúng là người mà họ khai báo. Đây là bước kiểm tra đầu vào, là "cổng bảo vệ" đầu tiên.
- **Mở rộng ví dụ thực tế:** Tòa nhà văn phòng an ninh cao Khi bạn đến một tòa nhà

văn phòng, quy trình xác thực không chỉ là trình chứng minh nhân dân. Nó có thể bao gồm:

1. Bạn xuất trình thẻ nhân viên có ảnh và tên của bạn.
2. Bảo vệ đối chiếu ảnh trên thẻ với khuôn mặt của bạn.
3. Họ có thể yêu cầu bạn quẹt thẻ vào một đầu đọc để xác nhận thẻ còn hiệu lực trong hệ thống.

Sau khi hoàn tất cả ba bước trên, người bảo vệ mới tin tưởng **bạn chính là nhân viên X** của công ty. Toàn bộ quá trình đó chính là Authentication.

– **Trong ứng dụng web, quá trình này diễn ra như thế nào?** Đây là quá trình người dùng cung cấp "bằng chứng" về danh tính của họ.

1. **Người dùng cung cấp thông tin:** Phổ biến nhất là nhập username và password vào một form đăng nhập.
2. **Hệ thống kiểm tra:** Server nhận thông tin này. Nó sẽ **hash** mật khẩu người dùng nhập vào và so sánh với giá trị hash đã được lưu trữ an toàn trong cơ sở dữ liệu. **Lưu ý quan trọng:** Hệ thống không bao giờ lưu mật khẩu dưới dạng văn bản thuần (plain text).
3. Kết quả xác thực:
 - **Thành công:** Nếu thông tin khớp, hệ thống xác nhận danh tính người dùng. Nó sẽ tạo ra một "bằng chứng kỹ thuật số" (digital proof) để ghi nhớ trạng thái đăng nhập này cho các yêu cầu sau. Bằng chứng này thường là một **session cookie** lưu trên trình duyệt hoặc một **JSON Web Token (JWT)** được trả về cho client.
 - **Thất bại:** Nếu không khớp, hệ thống từ chối truy cập và trả về thông báo lỗi "Sai tên đăng nhập hoặc mật khẩu".

– **Các hình thức xác thực phổ biến:**

- **Thứ bạn biết (Something you know):** Mật khẩu, mã PIN, câu hỏi bí mật.
- **Thứ bạn có (Something you have):** Điện thoại để nhận mã OTP, thẻ smart card, USB token (khóa bảo mật YubiKey).
- **Thứ bạn là (Something you are):** Dữ liệu sinh trắc học như vân tay, khuôn mặt, võng mạc.

- **Xác thực đa yếu tố (Multi-Factor Authentication - MFA):** Kết hợp từ hai yếu tố trở lên (ví dụ: mật khẩu + mã OTP) để tăng cường bảo mật.

10.1.2. Authorization (Phân quyền) - "Bạn được phép làm gì?"

- **Là gì?** Sau khi danh tính của một người dùng đã được xác thực thành công, Authorization là quá trình kiểm tra **quyền hạn** của người dùng đó để quyết định họ có được phép truy cập một tài nguyên hay thực hiện một hành động cụ thể hay không. Đây là "cổng kiểm soát" ở các lớp sâu hơn bên trong.
- **Tiếp tục ví dụ thực tế: Tòa nhà văn phòng** Sau khi bảo vệ xác thực bạn là nhân viên X, bạn được cấp một chiếc thẻ ra vào (access card). Chiếc thẻ này chính là "bằng chứng" đã xác thực của bạn. Bây giờ:
 - Bạn quẹt thẻ để đi qua cửa chính vào sảnh. **Được phép.**
 - Bạn quẹt thẻ để vào thang máy lên tầng 10, nơi phòng Kinh doanh làm việc. **Được phép.**
 - Bạn dùng thẻ để lên tầng 15, khu vực Ban Giám đốc. Thẻ báo lỗi. **Bị từ chối.**
 - Bạn thử dùng thẻ để mở cửa phòng server. Thẻ báo lỗi. **Bị từ chối.**

Hệ thống cửa và thang máy không cần biết bạn là ai nữa (việc đó bảo vệ đã làm), nó chỉ cần đọc thông tin quyền hạn được lập trình sẵn trên thẻ của bạn. Đây chính là Authorization. Một "Trưởng phòng IT" cũng đã được xác thực, nhưng thẻ của họ sẽ mở được cửa phòng server.

- **Trong ứng dụng web, quá trình này diễn ra như thế nào?** Sau khi bạn đăng nhập thành công (đã có cookie hoặc JWT), mỗi khi bạn thực hiện một yêu cầu mới (ví dụ: truy cập một URL, nhấn một nút), hệ thống sẽ thực hiện phân quyền:
 1. **Hệ thống nhận dạng bạn:** Dựa vào cookie hoặc token bạn gửi kèm theo request, hệ thống biết bạn là người dùng đã đăng nhập.
 2. **Hệ thống kiểm tra quyền:** Nó sẽ kiểm tra các quyền hạn gắn liền với tài khoản của bạn. Các quyền này thường được định nghĩa qua:
 - **Vai trò (Roles):** Mô hình phổ biến nhất. Ví dụ: tài khoản của bạn có vai trò là "Admin" hay chỉ là "Member"?
 - **Chính sách (Policies):** Các quy tắc phức tạp hơn. Ví dụ: một chính sách

"CanEditFinancialReport" có thể yêu cầu người dùng phải thuộc vai trò "Accountant" VÀ phải truy cập từ IP nội bộ của công ty.

3. Kết quả phân quyền:

- **Thành công:** Bạn được phép truy cập trang /Admin/Dashboard, được phép thấy nút "Xóa sản phẩm" và thực hiện hành động xóa.
- **Thất bại:** Khi bạn cố truy cập /Admin/Dashboard với tài khoản "Member", hệ thống sẽ trả về lỗi **403 Forbidden** (Bạn không có quyền) hoặc chuyển hướng bạn về trang chủ. Nút "Xóa sản phẩm" cũng sẽ bị ẩn đi trên giao diện.

– So sánh và Mối quan hệ cốt lõi

Ghi nhớ: Authentication luôn xảy ra trước Authorization. Bạn phải biết người đó là ai trước khi có thể quyết định họ được làm gì. Không thể cấp quyền cho một người vô danh.

Tiêu chí	Authentication (Xác thực)	Authorization (Phân quyền)
Câu hỏi trả lời	"Bạn là ai?"	"Bạn được phép làm gì?"
Mục đích	Xác minh danh tính người dùng.	Xác minh quyền truy cập của người dùng.
Ví dụ	Đăng nhập bằng username/password, xác thực vân tay.	Kiểm tra vai trò "Admin", kiểm tra quyền "edit_post".
Kết quả	Tạo ra một định danh (identity) đã được xác thực (cookie, token).	Quyết định cho phép (Allow) hoặc từ chối (Deny) truy cập.
Trạng thái	Là một quá trình thường chỉ diễn ra một lần (lúc đăng nhập).	Là một quá trình diễn ra liên tục với mỗi yêu cầu truy cập tài nguyên.

Đây là hai khái niệm nền tảng của bảo mật, thường bị nhầm lẫn với nhau.

10.2. ASP.NET Core Identity - "Bộ phận An ninh & Nhân sự" dựng sẵn

Việc tự xây dựng một hệ thống xác thực và phân quyền từ đầu là một "bãi mìn" đầy rủi ro. Bạn phải xử lý việc băm và lưu trữ mật khẩu an toàn, quản lý session/cookie, chống lại các cuộc tấn công như Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), và còn nhiều hơn nữa. Bất kỳ sai sót nhỏ nào cũng có thể dẫn đến lỗ hổng bảo mật nghiêm trọng.

Để giải quyết vấn đề này, Microsoft cung cấp một giải pháp "chìa khóa trao tay" mạnh mẽ: **ASP.NET Core Identity**. Đây là một nền tảng bảo mật cấp doanh nghiệp, được tích hợp sẵn vào ASP.NET Core, giúp bạn giải quyết tất cả các bài toán trên một cách chuyên nghiệp và an toàn.

10.2.1. Identity là gì?

Hãy coi ASP.NET Core Identity như một "**bộ phận An ninh & Nhân sự**" được cài đặt sẵn vào ứng dụng của bạn ngay từ ngày đầu. Thay vì bạn phải tự tuyển dụng và đào tạo, bộ phận này đã có sẵn một đội ngũ chuyên nghiệp để thực hiện các nghiệp vụ cốt lõi:

- **Quản lý người dùng (User Management):** Giống như phòng Nhân sự quản lý hồ sơ nhân viên, Identity cung cấp API để quản lý toàn bộ vòng đời của một người dùng: tạo tài khoản mới, tìm kiếm, cập nhật thông tin (email, số điện thoại), và xóa tài khoản.
- **Quản lý mật khẩu (Password Management):** Đây là nghiệp vụ tối quan trọng. Identity tự động **băm và salt mật khẩu** (hashing and salting) trước khi lưu vào CSDL. Điều này đảm bảo rằng ngay cả khi CSDL bị rò rỉ, kẻ tấn công cũng không thể dịch ngược ra mật khẩu gốc. Nó cũng cung cấp sẵn các tính năng như đổi mật khẩu, quên mật khẩu và đặt lại mật khẩu.
- **Xác thực người dùng (Authentication):** Giống như đội an ninh ở cổng kiểm tra thẻ, Identity xử lý việc xác thực thông tin đăng nhập. Khi thành công, nó sẽ tạo và quản lý **cookie xác thực** một cách an toàn để duy trì trạng thái đăng nhập của người dùng qua các request khác nhau.
- Quản lý vai trò (Roles) và quyền hạn (Claims):
 - **Roles:** Cho phép bạn nhóm người dùng vào các vai trò như "Admin",

"Manager", "Member". Đây là cách phân quyền đơn giản và phổ biến.

- **Claims:** Cung cấp một cơ chế phân quyền linh hoạt và chi tiết hơn. Một claim là một cặp thông tin key-value về người dùng (ví dụ: department = "IT", employeeId = "123"). Bạn có thể cấp quyền dựa trên các claim này.
- **Hỗ trợ nâng cao:** Identity còn tích hợp sẵn các tính năng phức tạp như xác thực hai yếu tố (2FA) qua email hoặc ứng dụng authenticator, và cho phép người dùng đăng nhập bằng các tài khoản mạng xã hội như Google, Facebook, Microsoft mà không cần tạo tài khoản mới trên hệ thống của bạn.

10.2.2. Các thành phần chính - "Nhân sự" trong bộ máy

"Bộ phận An ninh & Nhân sự" này có một cơ cấu tổ chức rõ ràng với các "nhân sự" (các class) chuyên trách:

– IdentityUser - Hồ sơ nhân viên

- Đây là class đại diện cho một người dùng, tương đương một bộ hồ sơ nhân viên. Nó chứa các thuộc tính cơ bản nhất được định nghĩa sẵn như Id, UserName, NormalizedUserName, Email, NormalizedEmail, EmailConfirmed, PasswordHash, PhoneNumber.
- **Ví dụ thực tế:** Thông thường, bạn sẽ cần thêm các thông tin riêng. Bạn có thể dễ dàng làm điều này bằng cách kế thừa:

```
//C#  
// Trong file Models/ApplicationUser.cs  
public class ApplicationUser : IdentityUser  
{  
    public string? FullName { get; set; }  
    public DateTime DateOfBirth { get; set; }  
    // Thêm các thuộc tính khác bạn cần...  
}
```

– IdentityRole - Chức danh

- Class này đại diện cho một vai trò trong hệ thống, ví dụ như "Admin", "Manager", "Member". Nó rất đơn giản, chủ yếu chứa Id và Name của vai trò.

– UserManager<TUser> - Trưởng phòng Nhân sự

- Đây là lớp dịch vụ trung tâm để thực hiện các thao tác liên quan đến **quản lý**

người dùng. Bạn sẽ sử dụng nó để tạo, tìm, sửa, xóa người dùng và gán vai trò cho họ. Nó là "Trưởng phòng Nhân sự" vì nó xử lý mọi nghiệp vụ liên quan đến "hồ sơ".

- Ví dụ sử dụng:

```
//C#
// Giả sử _userManager đã được inject vào Controller
var user = new ApplicationUser { UserName = "testuser@example.com", Email = "testuser@example.com", FullName = "Test User" };
// Tạo người dùng mới với mật khẩu
var result = await _userManager.CreateAsync(user, "Password@123");
if (result.Succeeded)
{
    // Gán vai trò "Member" cho người dùng vừa tạo
    await _userManager.AddToRoleAsync(user, "Member");
}
```

– SignInManager<TUser> - Nhân viên An ninh cổng

- Lớp dịch vụ này chuyên xử lý các nghiệp vụ **xác thực**. Nó không quan tâm đến việc tạo hay sửa người dùng, mà chỉ tập trung vào việc kiểm tra thông tin đăng nhập, đăng xuất, và quản lý trạng thái đăng nhập. Nó chính là "Nhân viên An ninh" đứng ở cổng.

- Ví dụ sử dụng:

```
//C#
// Giả sử _signInManager đã được inject vào Controller
// Thử đăng nhập cho người dùng
var result = await
_signInManager.PasswordSignInAsync("testuser@example.com", "Password@123",
isPersistent: false, lockoutOnFailure: false);
if (result.Succeeded)
{
    // Đăng nhập thành công, chuyển hướng đến trang chủ
    return LocalRedirect(returnUrl);
}
else
{
    // Đăng nhập thất bại, hiển thị lỗi
    ModelState.AddModelError(string.Empty, "Invalid login attempt.");
    return Page();
}
```

– **IdentityDbContext - Tủ hồ sơ**

- Đây là một phiên bản DbContext của Entity Framework Core được cấu hình sẵn để làm việc với tất cả các bảng CSDL cần thiết cho Identity. Khi bạn thêm Identity vào dự án, nó sẽ tự động tạo ra các bảng như:AspNetUsers, AspNetRoles, AspNetUserRoles, AspNetUserClaims,... Bạn không cần phải tự định nghĩa các bảng này.

10.2.3. Scaffolding - "Kiến trúc sư & Đội thi công tự động"

Scaffolding là một công cụ sinh mã nguồn tự động cực kỳ mạnh mẽ. Thay vì phải tự tay tạo ra hàng chục file Razor View/Page và các lớp PageModel/Controller cho các chức năng như:

- Đăng ký (Register.cshtml); Đăng nhập (Login.cshtml); Đăng xuất
- Quản lý thông tin cá nhân (Manage/Index.cshtml)
- Đổi mật khẩu (Manage/ChangePassword.cshtml)
- Quên mật khẩu (ForgotPassword.cshtml)

Bạn chỉ cần chạy một vài câu lệnh. Công cụ Scaffolding sẽ hoạt động như một "đội thi công" dựa trên "bản thiết kế" có sẵn, tự động tạo ra tất cả các file này và đặt chúng vào đúng vị trí trong dự án của bạn (thường là trong thư mục /Areas/Identity/Pages).

– Lợi ích lớn nhất:

- **Tốc độ:** Bạn có một hệ thống xác thực hoàn chỉnh chỉ trong vài phút.
- **Tính tùy chỉnh:** Điều quan trọng là các file được sinh ra là **của bạn**. Bạn có toàn quyền mở chúng ra, chỉnh sửa giao diện HTML để phù hợp với thiết kế của website (ví dụ: thêm logo, đổi màu sắc), hoặc thêm các logic nghiệp vụ riêng vào phần code-behind (.cshtml.cs). Ví dụ, sau khi người dùng đăng ký thành công, bạn có thể thêm logic để gửi một email chào mừng.

10.3. Bảo vệ tài nguyên - "Người bảo vệ" và "Thẻ VIP" guarding

Sau khi đã có hệ thống Identity để xác thực và quản lý người dùng, bước tiếp theo là sử dụng nó để bảo vệ các khu vực nhạy cảm trong ứng dụng. Chúng ta sẽ triển khai các "chốt an ninh" để đảm bảo chỉ những người có quyền mới có thể truy cập.

10.3.1. Cookie-based Authentication

Đối với các ứng dụng web truyền thống (MVC, Razor Pages), cơ chế xác thực mặc định và phổ biến nhất mà ASP.NET Core Identity sử dụng là dựa trên **Cookie**.

– **Tương tự "Vòng tay sự kiện":** Hãy tưởng tượng bạn tham dự một sự kiện.

1. **Soát vé (Đăng nhập):** Tại cổng vào, bạn xuất trình vé (username & password). Nhân viên an ninh xác minh vé của bạn là hợp lệ.
2. **Đeo vòng tay (Tạo Cookie):** Sau khi xác minh, họ đeo cho bạn một chiếc vòng tay có mã vạch đặc biệt. Chiếc vòng tay này là bằng chứng bạn đã được kiểm tra và được phép vào trong.
3. **Đi lại tự do (Gửi request):** Bây giờ, bạn có thể đi lại giữa các sân khấu, khu ẩm thực... Tại mỗi khu vực có kiểm soát, thay vì phải lôi vé ra lần nữa, bạn chỉ cần giơ vòng tay ra. Nhân viên an ninh sẽ quét mã vạch trên vòng tay để xác nhận bạn là khách hợp lệ.
4. **Hết hạn:** Vòng tay này chỉ có giá trị trong ngày diễn ra sự kiện. Sang ngày hôm sau, nó sẽ không còn hợp lệ nữa.

– **Luồng hoạt động kỹ thuật:**

1. **Người dùng đăng nhập thành công:** Bạn gọi `_signInManager.PasswordSignInAsync(...)` và nó trả về `Succeeded`.
2. **Server tạo và mã hóa Cookie:** `SignInManager` sẽ tạo một cookie. Bên trong cookie này không chỉ là một mã session ngẫu nhiên, mà nó chứa các thông tin quan trọng về người dùng (gọi là **claims**) đã được mã hóa, ví dụ:
 - User ID
 - User Name
 - Các vai trò (Roles) của người dùng (ví dụ: "Admin", "Member")
 - Thời gian hết hạn của cookie
3. **Gửi Cookie về trình duyệt:** Server gửi cookie này trong phần header của HTTP response. Trình duyệt tự động lưu nó lại.
4. **Tự động đính kèm Cookie:** Từ đó, với **mọi request** gửi đến cùng một domain, trình duyệt sẽ tự động đính kèm cookie này vào HTTP request header.

5. **Server giải mã và xác thực:** Khi nhận được request, middleware xác thực của ASP.NET Core sẽ đọc cookie, giải mã nó, và tái tạo lại danh tính của người dùng. Dựa vào thông tin này, nó biết chính xác người dùng là ai, có vai trò gì, mà không cần phải truy vấn lại CSDL mỗi lần.

10.3.2. Attribute [Authorize] - "Người bảo vệ" ở cửa

Đây là công cụ chính để bạn đặt các "chốt an ninh". Nó là một attribute cực kỳ đơn giản nhưng vô cùng mạnh mẽ mà bạn có thể đặt ngay trên một Controller hoặc một Action riêng lẻ.

- **Khi đặt trên toàn bộ Controller:** Hành động này giống như bạn đặt một người bảo vệ ngay tại cửa chính của một khu vực (ví dụ: khu vực quản trị). Bất kỳ ai muốn vào bất kỳ phòng nào bên trong khu vực này đều phải được kiểm tra trước.
- Ví dụ minh họa:

```
//C#
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Authorization; // Phải thêm using này
[Authorize] // <-- BẢO VỆ ĐẶT Ở ĐÂY
public class AdminController : Controller
{
    // Yêu cầu phải đăng nhập
    public IActionResult Dashboard()
    {
        return View();
    }

    // Cũng yêu cầu phải đăng nhập
    public IActionResult ManageUsers()
    {
        return View();
    }
}
```

- **Kết quả:** Nếu một người dùng chưa đăng nhập cố gắng truy cập vào URL /Admin/Dashboard hoặc /Admin/ManageUsers, họ sẽ ngay lập tức bị "chặn" lại và tự động chuyển hướng đến trang đăng nhập. Sau khi đăng nhập thành công, hệ thống sẽ tự động đưa họ trở lại trang mà họ đã cố gắng truy cập.

- **Khi đặt trên một Action cụ thể:** Hành động này giống như bạn chỉ đặt bảo vệ ở cửa một căn phòng quan trọng, trong khi các phòng khác trong cùng một hành lang thì ai cũng có thể vào.
- Ví dụ minh họa:

```
//C#
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Authorization;
public class ProductController : Controller
{
    // Ai cũng có thể xem danh sách sản phẩm
    public IActionResult Index()
    {    // ... code lấy tất cả sản phẩm
        return View();
    }
    // Ai cũng có thể xem chi tiết một sản phẩm
    public IActionResult Details(int id)
    {    // ... code lấy sản phẩm theo id
        return View();
    }
    [Authorize] // <-- BẢO VỆ CHỈ ĐẶT Ở PHÒNG NÀY
    public IActionResult MyOrders()
    {
        // Chỉ người dùng đã đăng nhập mới có thể xem đơn hàng của họ
        // ... code lấy đơn hàng của người dùng hiện tại
        return View();
    }
}
```

- **Kết quả:** Tất cả người dùng (kể cả khách vãng lai) đều có thể xem trang /Product và /Product/Details/5. Nhưng khi họ nhấn vào link /Product/MyOrders, họ sẽ bị yêu cầu đăng nhập.

10.3.3.Role-based Authorization - "Kiểm tra thẻ VIP"

Chỉ kiểm tra xem người dùng đã đăng nhập hay chưa (Authentication) là không đủ. Chúng ta cần kiểm tra xem họ có **quyền hạn** (Authorization) để vào một khu vực cụ thể hay không. Đây là lúc việc phân quyền dựa trên vai trò (Role-based Authorization) phát huy tác dụng.

- **Cú pháp:** Nó là một phiên bản nâng cấp của [Authorize]. Bạn chỉ cần truyền thêm tham số Roles.
- Ví dụ minh họa:

```
//C#
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Authorization;

// "Người bảo vệ" này giờ đây được trang bị thêm một danh sách khách VIP
[Authorize(Roles = "Admin")]
public class AdminController : Controller
{
    public IActionResult Dashboard()
    {
        return View();
    }
}
```

- **Nó làm gì?** "Người bảo vệ" này giờ đây sẽ thực hiện một quy trình kiểm tra 2 bước nghiêm ngặt hơn:
 1. **Kiểm tra "vòng tay" (Authentication):** Người dùng đã đăng nhập chưa? Nếu chưa, chuyển đến trang đăng nhập.
 2. **Kiểm tra "thẻ VIP" (Authorization):** Nếu đã đăng nhập, người bảo vệ sẽ kiểm tra thông tin trên "vòng tay" (cookie) xem có ghi vai trò là "Admin" không.
 - o Nếu có, cho phép truy cập.
 - o Nếu không (ví dụ: vai trò của họ chỉ là "Member"), người dùng sẽ bị từ chối và thường sẽ thấy một trang báo lỗi "Access Denied" (Truy cập bị từ chối - Lỗi 403).
- **Chỉ định nhiều vai trò (OR Logic):** Bạn có thể cho phép nhiều loại "thẻ VIP" khác nhau cùng vào cửa. Người dùng chỉ cần có **một trong số** các vai trò được liệt kê là đủ.
- Ví dụ minh họa:

```
//C#
// Cho phép người dùng có vai trò "Admin" HOẶC "Manager" truy cập
[Authorize(Roles = "Admin,Manager")]
```

```
public class ReportController : Controller
{
    public IActionResult MonthlySalesReport()
    {
        return View();
    }
}
```

Trong ví dụ này, một người dùng đăng nhập với vai trò "Manager" sẽ truy cập thành công, tương tự với vai trò "Admin". Nhưng một người dùng với vai trò "Member" sẽ bị chặn lại.

10.4. Hướng dẫn thực hành

10.4.1. Cách 1: Visual Studio (Giao diện trực quan)

1. Mở cửa sổ Scaffolding:

- Trong **Solution Explorer**, chuột phải vào project SportsStore.WebUI -> **Add -> New Scaffolded Item....**
- Chọn **Identity** trong danh sách bên trái, rồi chọn **Identity** ở giữa và nhấn **Add**.

2. Cấu hình Identity: *

- **Override all files: Tick vào ô này.** Điều này sẽ tạo ra tất cả các file Razor Pages (.cshtml và .cshtml.cs) vào dự án của bạn, cho phép bạn tùy chỉnh giao diện sau này.
- **Data context class:** Nhấn nút +. Cực kỳ quan trọng, hãy chọn project chứa CSDL của bạn (SportsStore.Infrastructure) và đặt tên cho DbContext mới, ví dụ AppIdentityDbContext. **Không dùng chung ApplicationDbContext** để tách biệt dữ liệu ứng dụng và dữ liệu người dùng.
- **User class:** Nhấn nút +. Tương tự, chọn project SportsStore.Domain và đặt tên, ví dụ AppUser.
- Nhấn **Add**. Visual Studio sẽ tự động cài các gói NuGet cần thiết, tạo các class, các trang Razor, và cập nhật Program.cs cho bạn.

10.4.2. Cách 2: VS Code + .NET CLI (Dòng lệnh)

1. Cài đặt công cụ Code Generator (nếu chưa có):

```
//Bash
# Cài đặt công cụ này một lần duy nhất trên máy của bạn
dotnet tool install -g dotnet-aspnet-codegenerator
```

2. Chạy lệnh Scaffolding:

- Mở terminal tại thư mục của project SportsStore.WebUI.

```
//Bash
dotnet aspnet-codegenerator identity -dc
SportsStore.Infrastructure.AppIdentityDbContext --files
"Account.Register;Account.Login;Account.Logout"
```

- dotnet aspnet-codegenerator identity: Gọi công cụ và chỉ định rằng chúng ta muốn scaffold Identity.
- -dc SportsStore.Infrastructure.AppIdentityDbContext: Cờ -dc (--dbContext) chỉ định tên đầy đủ (bao gồm cả namespace) của DbContext sẽ được tạo hoặc sử dụng.
- --files "...": Chỉ định danh sách các trang bạn muốn tạo. Để tạo tất cả, bạn có thể bỏ qua cờ này.

10.4.3. Các bước chung sau khi Scaffold

1. Tạo và áp dụng Migration cho Identity:

- Identity sử dụng DbContext riêng (AppIdentityDbContext), vì vậy chúng ta cần một bộ migration riêng cho nó.
- **PMC (VS):**

```
//PowerShell
Add-Migration IdentityInitial -Context AppIdentityDbContext
Update-Database -Context AppIdentityDbContext
```

- **.NET CLI (VS Code):**

```
//Bash
dotnet ef migrations add IdentityInitial -p SportsStore.Infrastructure -c
AppIdentityDbContext
dotnet ef database update -p SportsStore.Infrastructure -c
AppIdentityDbContext
```

- -Context / -c: Chỉ định DbContext nào sẽ được sử dụng cho migration này.

- -p: Chỉ định project chứa DbContext.

2. Seed dữ liệu Admin:

- Tạo một file IdentitySeedData.cs mới trong SportsStore.Infrastructure.
- **Code:**

```
//C#
using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Identity;
using Microsoft.Extensions.DependencyInjection;
using SportsStore.Domain; // Nơi chứa AppUser

public static class IdentitySeedData {
    public static async Task EnsurePopulated(IApplicationBuilder app) {
        // Lấy các dịch vụ UserManager và RoleManager từ DI Container
        var scope = app.ApplicationServices.CreateScope();
        UserManager<AppUser> userManager = scope.ServiceProvider
            .GetRequiredService<UserManager<AppUser>>();
        RoleManager<IdentityRole> roleManager = scope.ServiceProvider
            .GetRequiredService<RoleManager<IdentityRole>>();

        string roleName = "Admin";
        string userName = "admin@example.com";
        string password = "MySecretPassword123$";
        // 1. Tạo vai trò "Admin" nếu nó chưa tồn tại
        if (await roleManager.FindByNameAsync(roleName) == null) {
            await roleManager.CreateAsync(new IdentityRole(roleName));
        }
        // 2. Tạo người dùng "admin" nếu chưa tồn tại
        if (await userManager.FindByNameAsync(userName) == null) {
            AppUser user = new AppUser { UserName = userName, Email =
userName };
            var result = await userManager.CreateAsync(user, password);

            // 3. Gán người dùng đó vào vai trò "Admin"
            if (result.Succeeded) {
                await userManager.AddToRoleAsync(user, roleName);
            }
        }
    }
}
```

- Gọi Seeder trong Program.cs:

```
//C#  
// ...  
SeedData.EnsurePopulated(app);  
  
// Chờ cho seeder hoàn thành trước khi chạy app  
await IdentitySeedData.EnsurePopulated(app);  
  
app.Run();
```

CÂU HỎI LÝ THUYẾT

1. Một người dùng đã đăng nhập thành công vào hệ thống. Khi họ cố gắng truy cập một chức năng chỉ dành cho Admin, hệ thống đã từ chối. Đây là ví dụ của Authentication hay Authorization?
2. Trong ASP.NET Core Identity, lớp dịch vụ nào chịu trách nhiệm tạo người dùng mới (UserManager hay SignInManager)?
3. Khi bạn chạy scaffolding Identity, nó sẽ tự động cập nhật file nào để đăng ký các dịch vụ cần thiết?
4. Để bảo vệ một Action chỉ cho phép người dùng có vai trò "Editor" hoặc "Publisher" truy cập, bạn sẽ viết attribute [Authorize] như thế nào?

TỔNG KẾT BÀI HỌC

Bài này, chúng ta đã "gia cố" ứng dụng của mình bằng một hệ thống bảo mật chuyên nghiệp:

- Chúng ta đã phân biệt rõ ràng hai khái niệm nền tảng là **Authentication** và **Authorization**.
- Chúng ta đã tích hợp **ASP.NET Core Identity**, một framework mạnh mẽ và an toàn, giúp tiết kiệm hàng Bài, thậm chí hàng tháng phát triển.
- Chúng ta đã học cách "khóa" các tài nguyên nhạy cảm bằng **[Authorize]** và phân quyền dựa trên **vai trò (Roles)**.

Khu vực quản trị của bạn giờ đây đã được bảo vệ, chỉ những người có phận sự mới có thể vào. Đây là một bước tiến lớn để đưa ứng dụng của bạn đến gần hơn với môi trường thực tế.

BÀI TẬP VỀ NHÀ

1. Hoàn thiện chức năng Logout:

- Công cụ scaffolding đã tạo sẵn trang Logout. Hãy tìm đến file `_Layout.cshtml` và thêm một đoạn code để hiển thị link "Login" nếu người dùng chưa đăng nhập, và hiển thị "Hello [Tên người dùng]!" cùng link "Logout" nếu đã đăng nhập. (Gợi ý: sử dụng `@inject SignInManager<AppUser> SignInManager` và `@inject UserManager<AppUser> UserManager` ở đầu file View để kiểm tra `SignInManager.IsSignedIn(User)`).

2. Tạo vai trò mới:

- Sửa file `IdentitySeedData.cs` để tạo thêm một vai trò mới tên là "Manager".
- Tạo thêm một người dùng mới (ví dụ: `manager@example.com`) và gán người dùng này vào cả hai vai trò "Manager" và "Member" (tạo thêm vai trò "Member" nếu cần).
- (Nâng cao) Tạo một `ManagerController` mới và bảo vệ nó với `[Authorize(Roles = "Manager")]`.

TÀI LIỆU THAM KHẢO

- [1]. **Introduction to Identity on ASP.NET Core:** [Microsoft Docs - Introduction to Identity](#)
- [2]. **Scaffold Identity in ASP.NET Core projects:** [Microsoft Docs - Scaffold Identity](#)
- [3]. **Role-based authorization in ASP.NET Core:** [Microsoft Docs - Role-based Authorization](#)

GỢI Ý TRẢ LỜI CÂU HỎI

1. Phân biệt Authentication và Authorization

Câu hỏi: Một người dùng đã đăng nhập thành công vào hệ thống. Khi họ cố gắng truy cập một chức năng chỉ dành cho Admin, hệ thống đã từ chối. Đây là ví dụ của Authentication hay Authorization?

- **Gợi ý trả lời:** Đây là một ví dụ điển hình của Authorization (Phân quyền).
- **Authentication (Xác thực)** đã hoàn tất thành công ngay khi người dùng đăng nhập. Hệ thống đã xác minh được "họ là ai".
- **Authorization (Phân quyền)** xảy ra sau đó, khi hệ thống kiểm tra "họ được phép làm gì". Việc từ chối truy cập chức năng Admin cho thấy hệ thống đã xác định rằng người dùng này, mặc dù đã được xác thực, nhưng không có quyền hạn (vai trò) cần thiết để thực hiện hành động đó.

2. Vai trò của UserManager và SignInManager

Câu hỏi: Trong ASP.NET Core Identity, lớp dịch vụ nào chịu trách nhiệm tạo người dùng mới (UserManager hay SignInManager)?

- **Gợi ý trả lời:** Lớp dịch vụ chịu trách nhiệm tạo người dùng mới là **UserManager<TUser>**.
- **UserManager:** Đóng vai trò như "Trưởng phòng Nhân sự", chịu trách nhiệm quản lý toàn bộ vòng đời của một người dùng: tạo, sửa, xóa, tìm kiếm, đặt lại mật khẩu, và gán vai trò.
- **SignInManager:** Đóng vai trò như "Nhân viên An ninh" ở cổng, chỉ tập trung vào các nghiệp vụ xác thực: kiểm tra thông tin đăng nhập (PasswordSignInAsync), đăng xuất (SignInManager.SignOutAsync), và quản lý trạng thái đăng nhập.

3. Tác động của Scaffolding Identity

Câu hỏi: Khi bạn chạy scaffolding Identity, nó sẽ tự động cập nhật file nào để đăng ký các dịch vụ cần thiết?

- **Gợi ý trả lời:** Khi bạn chạy scaffolding Identity, nó sẽ tự động cập nhật file **Program.cs** (đối với .NET 6 trở lên) hoặc file **Startup.cs** (đối với các phiên bản cũ hơn).

Cụ thể, nó sẽ thêm các dòng mã cần thiết để đăng ký dịch vụ của Identity vào DI container, trông tương tự như sau:

```
// Trong Program.cs
builder.Services.AddDefaultIdentity<IdentityUser>(options =>
options.SignIn.RequireConfirmedAccount = true)
    .AddEntityFrameworkStores<ApplicationDbContext>();
```

Dòng mã này báo cho ứng dụng biết cách sử dụng Entity Framework để lưu trữ dữ liệu người dùng và cấu hình các dịch vụ mặc định của Identity.

4. Sử dụng Attribute [Authorize] với nhiều vai trò

Câu hỏi: Để bảo vệ một Action chỉ cho phép người dùng có vai trò "Editor" hoặc "Publisher" truy cập, bạn sẽ viết attribute [Authorize] như thế nào?

- **Gợi ý trả lời:** Bạn sẽ viết attribute [Authorize] bằng cách liệt kê các vai trò trong thuộc tính Roles, phân cách nhau bởi dấu phẩy.

```
[Authorize(Roles = "Editor, Publisher")]
public IActionResult CreateArticle()
{
    // ... code xử lý
    return View();
}
```

Dấu phẩy ở đây hoạt động như một toán tử logic **HOẶC (OR)**. "Người bảo vệ" sẽ cho phép truy cập nếu người dùng đã đăng nhập và có vai trò là "Editor" **hoặc** có vai trò là "Publisher".